Experiment 10

Experiment to understand array function in Data science

Aim:

To understand array function in Data science.

Code:

```
import numpy as np

array = np.random.randint(1, 100, 9)
array

np.sqrt(array)

array.ndim

new_array = array.reshape(3, 3)
new_array

new_array.ndim

new_array.ravel()

newm = new_array.reshape(3, 3)
newm

newm[2, 1:3]

newm[1:2, 1:3]

new_array[0:3, 0:0]

new_array[0:2, 0:1]

new_array[0:3, 0:1]

new_array[1:3]
```

Output:

```
In [1]: array = np.random.randint(1, 100, 9)
In [2]: array
Out[2]: array([83, 25, 19, 47, 62, 15, 96, 39, 51])

In [3]: np.sqrt(array)
Out[3]: array([9.11043358, 5.        , 4.35889894, 6.8556546 , 7.87400787,
       3.87298335, 9.79795897, 6.244998  , 7.14142843])

In [4]: array.ndim
Out[4]: 1

In [5]: new_array = array.reshape(3, 3)
In [6]: new_array
Out[6]:
array([[83, 25, 19],
       [47, 62, 15],
       [96, 39, 51]])

In [7]: new_array.ndim
Out[7]: 2

In [8]: new_array.ravel()
Out[8]: array([83, 25, 19, 47, 62, 15, 96, 39, 51])

In [9]: newm = new_array.reshape(3, 3)
In [10]: newm[2, 1:3]
Out[10]: array([39, 51])

In [11]: newm[1:2, 1:3]
Out[11]: array([[62, 15]])

In [12]: new_array[0:3, 0:0]
Out[12]: array([], shape=(3, 0), dtype=int64)

In [13]: new_array[0:2, 0:1]
Out[13]:
array([[83],
       [47]])

In [14]: new_array[0:3, 0:1]
Out[14]:
array([[83],
       [47],
       [96]])

In [15]: new_array[1:3]
Out[15]:
array([[47, 62, 15],
       [96, 39, 51]])
```

Result:

NumPy array operations were successfully demonstrated. Arrays were created, reshaped from 1D to 2D, flattened using ravel(), and sliced across multiple dimensions. Element-wise square root computations validated NumPy's vectorized operation capabilities.